

Time-Off Microservice — The Plan

ExampleHR Engineering

2026-05-12

Planning Approach

Before writing a single line of code, we produced a Technical Requirement Document (TRD) that locked in every key decision. This forced clarity on the hard problems before the complexity of implementation obscured them.

The Central Insight: The Balance Model

The most important planning decision was defining exactly *what* “available balance” means and how to maintain it across two systems.

The invariant we chose:

`available_days = hcm_balance - pending_days`

where:

`hcm_balance` = what the HCM currently says is available
(updated only from HCM API responses, never computed locally)
`pending_days` = sum of days across all PENDING (not-yet-approved) requests
`available_days` = NEVER stored - always computed on read

This model solves the “two systems, one truth” problem cleanly:

- When a request is **created** > `pending_days` increases. HCM doesn’t know yet.
 - When a request is **approved** > HCM is told (deduct). `hcm_balance` decreases by the same amount that `pending_days` decreases. **Net effect on available_days: zero.** The balance was already “spoken for” at creation time.
 - When the HCM **changes a balance externally** (anniversary bonus) > `hcm_balance` is updated, `pending_days` is unchanged. `available_days` increases automatically.
-

Key Architecture Decisions

1. When to Submit to HCM

Decision: Submit to HCM on *approval*, not on creation.

Option	Pros	Why Rejected
Submit on creation	HCM always up to date	Manager may reject — wastes HCM API calls for every rejected request
Submit on approval	Only confirmed requests go to HCM	Slight delay; managed by local reservation
Always real-time	Authoritative	HCM downtime makes ExampleHR unreadable

Chosen: Submit on approval. Local reservation at creation time prevents over-allocation in ExampleHR without burdening the HCM with pending requests.

2. Concurrency Protection

Decision: Optimistic locking with `BEGIN IMMEDIATE` transactions.

SQLite is single-writer, but Node.js can context-switch between async operations. Two requests can both read the same balance row, both pass the balance check, and both try to reserve days — together exceeding the balance.

Protection mechanism:

Every `leave_balance` row has a version integer.

```
UPDATE ... SET pending_days += days, version = version + 1
      WHERE id = ? AND version = ?expected
```

```
If rows_affected == 0 > version changed (another request won)
    > retry with fresh data (up to 3 times)
```

All writes use `BEGIN IMMEDIATE` so SQLite acquires a write lock upfront, preventing the classic read-modify-write race condition.

3. HCM Call Placement

Decision: Real-time HCM `getBalance` call happens **OUTSIDE** the transaction.

If we held a SQLite write lock while waiting for an HTTP call to Workday (which might take 100–500ms), we’d serialize all concurrent requests unnecessarily. Instead:

1. Call HCM *before* opening the transaction (get authoritative balance)
2. Open `BEGIN IMMEDIATE`, do the optimistic-lock update
3. If the version check fails, retry — including a fresh HCM call

4. HCM Unavailability Handling

Decision: Graceful degradation with local shadow.

- HCM down during `getBalance` > use local shadow balance (less authoritative but safe)
- HCM down during `deductBalance` (approval) > return 502 Bad Gateway, request stays PENDING, no state mutation
- HCM down during `creditBalance` (cancel approved) > return 502, no state mutation

5. Mock HCM

Decision: A real, standalone NestJS server on port 3001 — not just HTTP mocks.

Reasoning: `nock` (HTTP interceptors) would only catch integration bugs. A real mock server with actual business logic (it enforces balance insufficiency, fires real webhook calls for anniversary bonuses) catches integration bugs AND behavioral contracts.

Data Model Plan

Five tables chosen to cleanly separate concerns:

Table	Purpose
<code>employees</code>	Reference data (thin — populated by HCM sync)
<code>locations</code>	Reference data
<code>leave_balances</code>	Shadow copy of HCM + pending tracking; the heart of the system
<code>time_off_requests</code>	Request lifecycle with state machine
<code>balance_audit_logs</code>	Append-only audit trail of every balance mutation

Key design choice on `leave_balances`: the `version` column for optimistic locking and `hcm_last_synced_at` for staleness awareness.

API Plan

The API was divided into three groups with different trust levels:

Group	Auth	Purpose
Employee/Manager endpoints	Authorization: Bearer <jwt>	Request lifecycle, balance reads
HCM sync endpoints	X-HCM-API-Key	Batch sync, single balance webhook
Mock HCM debug (test only)	None	PUT <code>/hcm/debug/balance</code> , POST <code>/hcm/debug/anniversary</code>

Test Strategy Plan

The assignment graded primarily on test rigor. The test plan had three layers:

Layer 1 — Unit tests (speed, coverage) Every service method tested in isolation with fully mocked dependencies. Target: catch logic bugs fast.

Layer 2 — Integration tests (correctness under HTTP) Full NestJS app with in-memory SQLite, HCM calls mocked vianock. Five mandatory regression scenarios: 1. Concurrent requests exceeding balance 2. Batch sync with anniversary bonus 3. Approve fails on HCM 422 4. Cancel approved — HCM credit called exactly once 5. Idempotency key deduplication

Layer 3 — E2E tests (behavioral contracts) Live mock HCM on port 3001. Tests that the *real* HTTP interaction between ExampleHR and HCM works end-to-end, including the anniversary webhook push.

Build Sequence

The implementation was planned as 12 sequential phases to ensure each layer was solid before building on top of it:

Phase	What
1	NestJS scaffold, TypeScript strict, Jest config
2	Database layer: TypeORM/SQLite, WAL, entities, migration, optimistic-lock helper
3	Common layer: filters, guards, interceptors, response envelope
4	HCM client with retry logic and typed error classes
5	Balance module: repository (all optimistic-lock mutations), service, controller
6	Time-off module: full state machine, balance algorithm, idempotency
7	HCM sync module: batch reconciliation, single webhook
8	Mock HCM server: in-memory state, debug API, anniversary push
9	Test infrastructure: factories, seeds, nock helpers
10	Integration tests: all 5 mandatory regressions
11	E2E tests: live mock HCM scenarios
12	Coverage validation and lint

Phases 3+4, 7+8, and 10+11 ran in parallel where there were no dependencies between them.

What Was Not Planned (Explicit Scope Cuts)

Out of Scope	Rationale
Outbox pattern for HCM submission failures	Over-engineering for take-home; documented as future improvement
JWT parsing / real RBAC	Stub sufficient for assignment
PostgreSQL	Assignment specified SQLite; repository pattern abstracts the difference
Leave policy rules	HCM's responsibility
Push notifications	Not in requirements